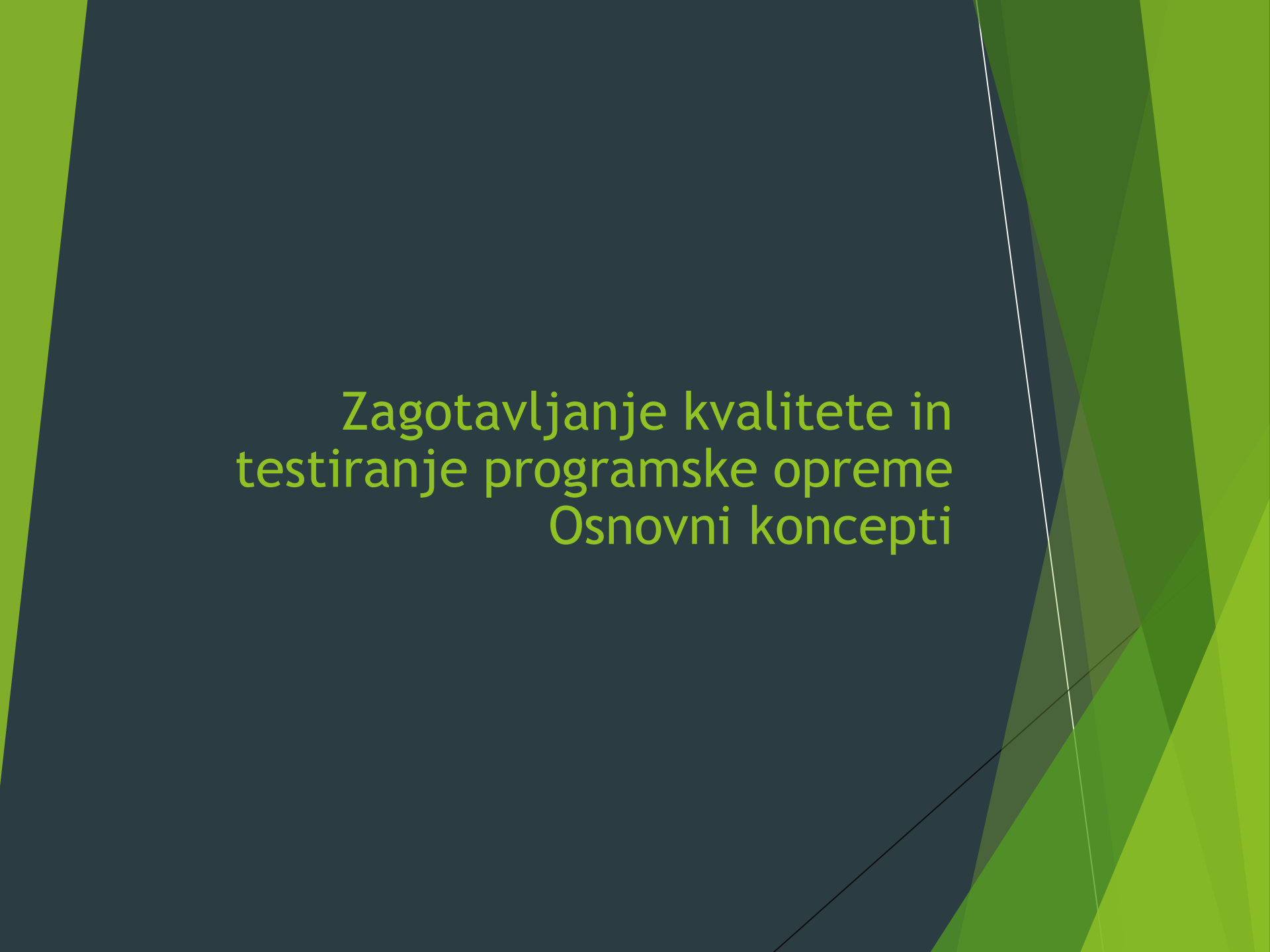




Zagotavljanje kvalitete in testiranje programske opreme Osnovni koncepti

Vsebina predmeta

Zgodovina preverjanja kvalitete
Kvaliteta programske opreme
Trendi
Pomen testiranja
Verifikacija in validacija
Napak, okvara, zmota in odpoved
Zanesljivost programske opreme
Cilji testiranja
Kaj je testni primer?
Pričakovani rezultati
Koncept celovitega testiranja
Osrednje vprašanje pri testiranju
Testne aktivnosti
Nivo testiranja
Viri podatkov za izbiro testnih metod
Testiranje po principu bele- in črne-škafle
Načrtovanje testiranja in testnih vzorcev
Opazovanje in merjenje izvajanja testov
Testna orodja in avtomatiziranje testiranja
Organizacija in vodenje testne skupine

Raziskave s področja kvalitete programske opreme - bibliografija

SCOPUS

Iskalni niz: Software AND quality
omenjeno na področje „Computer
science“

68354 člankov

Prvi članek 1954:

Traditional methods of teaching software engineering at the undergraduate and graduate levels lack full effectiveness due to the method of organization of the courses and compressed time schedules. This paper presents a studio approach to teaching large-scale, quality software development that uses the technique of reflective practice in a highly interactive learning environment over a sixteen-month period. The results of a prototype offering of this course are described and analyzed. ©

Zgodovina zagotavljanja kvalitete

Začetki na Japonskem v 40-ih letih - Deming, Juran in Ishikawa

V 50-ih letih Deming predstavi statistično preverjanje kvalitete Japonskim inženirjem

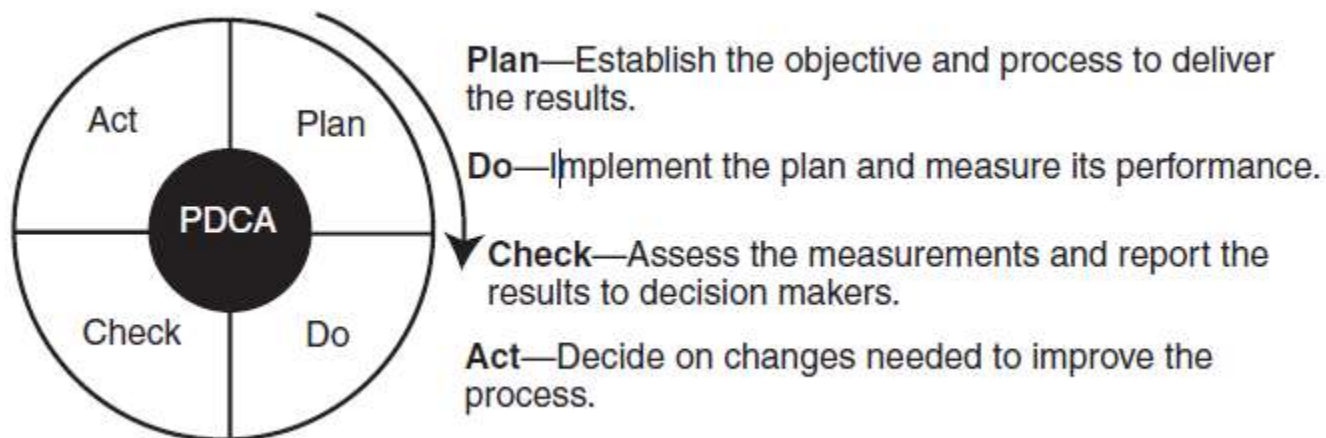
Statistična kontrola kvalitete (angl. SQC) je metoda, ki temelji na meritvah in statistiki

- SQC metode uporabljajo 7 osnovnih orodij za preverjanje kvalitete
- Pareto analizo, trend diagrame, diagrame poteka, histograme, diagrame raztresenosti (Scatter diagram), diagrame kontrole, diagrame vzrokov in posledic

Taiichi Ohno v Toyoti vpelje „Lean“ koncept

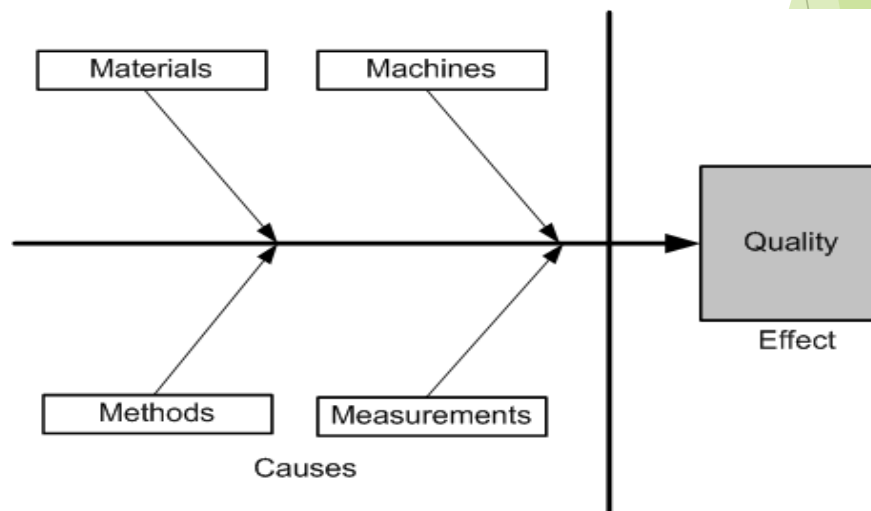
- Minimizacija porabe virov, ki ne doprinesejo k vrednosti produkta
- Skrajšati čas med naročilom in plačilom

- ▶ Leta 1950 Deming predstavi PDCA metodo, ki temelji na Shewhart-ovi metodi statistične kontrole kvalitete

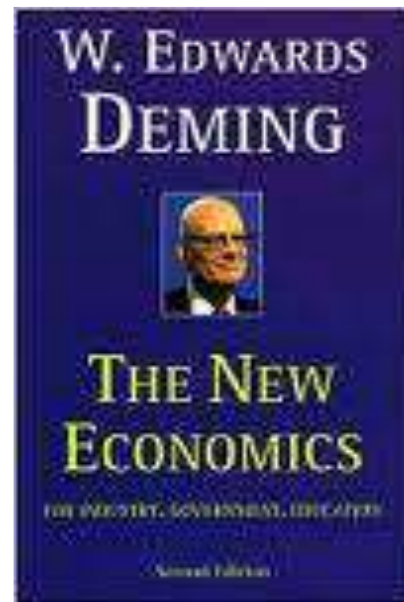
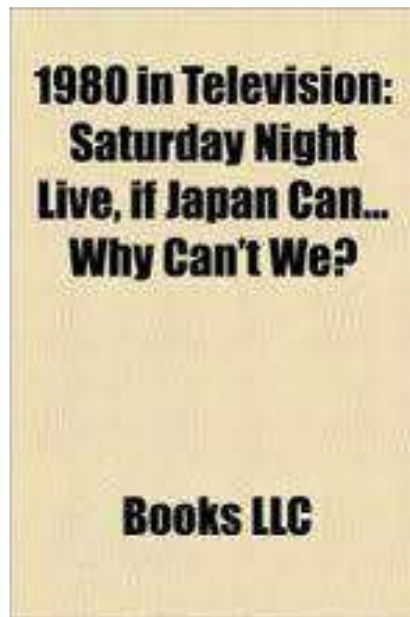
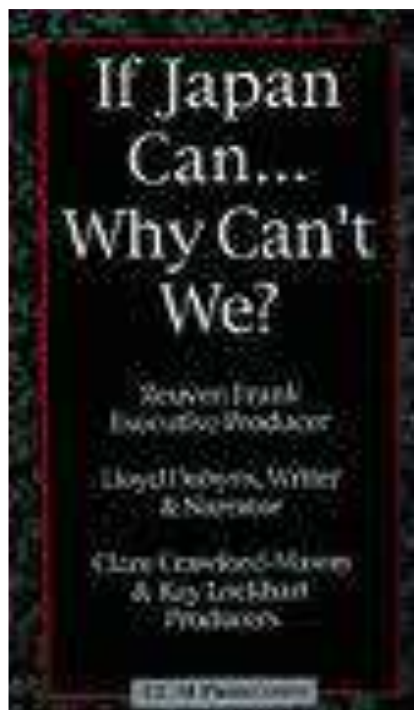


- ▶ Določitev ciljev
- ▶ Določitev izmerljivih mejnikov
- ▶ Ocenitev napredka glede na mejnike
- ▶ Sprejeti ukrepe za izboljšanje procesa v naslednjem ciklu

- ▶ Leta 1954, Joseph M. Juran predstavi metodo celovite kontrole kvalitete (TQC - Total Quality Control)
- ▶ Glavni elementi TQC so:
 - ▶ Najpomembnejša je kvaliteta, ne kratkoročni dobički
 - ▶ Najpomembnejša je stranka, ne proizvajalec
 - ▶ Odločitve se sprejemajo na podlagi dejstev in podatkov
 - ▶ Management vključuje in upošteva vse zaposlene
 - ▶ Management upravljajo odbori iz vseh področij
- ▶ V 60-ih Ishikawa predstavi inovativno metodologijo imenovano diagram vzrokov in posledic (Ishikawa diagram):



- ▶ Ishikawa iz statističnih podatkov ugotovi, da je disperzija kvalitete produkta odvisna od:
 - ▶ Materialov
 - ▶ Strojev
 - ▶ Metod
 - ▶ Meritev
- ▶ Ishikawa diagram spodbudi Japonske firme, da se za izboljšanje kvalitete osredotočijo na izboljšanje materialov, strojev, metod in meritev.



Kvaliteta programske opreme

- ▶ Kitchenham in Pfleeger podata 5 pogledov na kvaliteto programske opreme:
 - ▶ Transcendenten pogled
 - ▶ Kvaliteta je nekaj, kar se da prepoznati a se jo težko definira
 - ▶ Uporabniški pogled
 - ▶ Ali kvaliteta zadovoljuje uporabnikovim potrebam in zahtevam?
 - ▶ Proizvodni pogled
 - ▶ Kvaliteta v skladju s specifikacijami
 - ▶ Produktni pogled
 - ▶ Produktne karakteristike določajo celotno kvaliteto
 - ▶ Vrednosten pogled
 - ▶ Kvaliteta je odvisna od tega, koliko je kupec pripravljen plačati

Kvaliteta programske opreme z vidika dejavnikov in kriterijev kvalitete

- Dejavnik kvalitete predstavlja karakteristike obnašanja sistema
 - Npr.: pravilnost, zanesljivost, učinkovitost, testabilnost...
- Kriterij kvalitete je atribut, ki je povezan z razvojem programske opreme
 - Npr.: modularnost - atribut programske arhitekture

Modeli kvalitete

- Primeri: ISO 9126, CMM, TPI, TMM...

Pomen testiranja

Testiranje programske opreme
delimo v dve kategoriji:

- Statična analiza
 - Preverimo kodo in vzroke za vsa možna obnašanja sistema med delovanjem
 - npr.: pregled kode (Code review), pregled in analiza algoritmov
- Dinamična analiza
 - Zagon programa, da ugotovimo morebitne odpovedi
 - Opazovanje delovanja programa in ocenitev kvalitete

V praksi sta statična in dinamična
analiza komplementarni

Potrebno je združiti prednosti
obeh pristopov

Verifikacija in Validacija

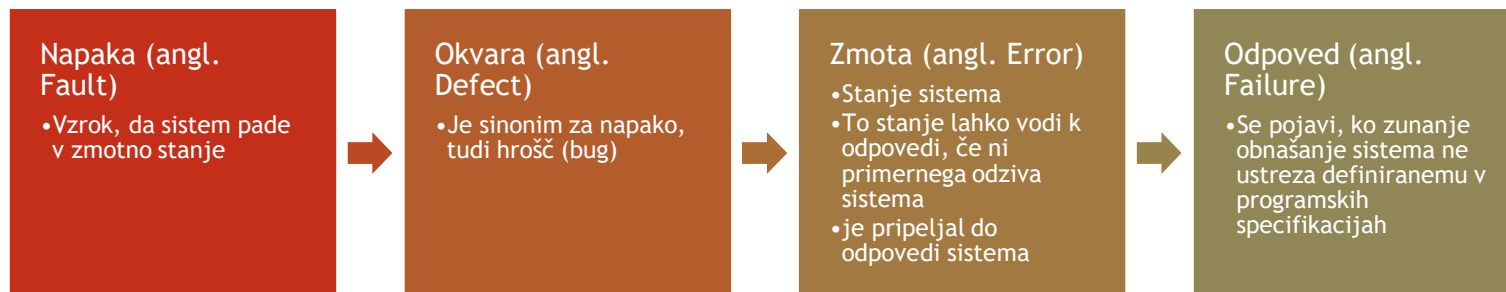
► Verifikacija

- Evaluacija programske opreme, s katero ugotovimo ali produkt v neki razvojni fazi izpolnjuje zahtevam, ki so bile postavljene pred začetkom te faze
 - Ali razvoj produkta poteka na pravilen način

► Validacija

- Evaluacija programske opreme, s katero ugotovimo ali produkt ustreza dejanskemu namenu
 - Ali razvijamo pravilen produkt

Napaka, okvara, zmota in odpoved



Zanesljivost programske opreme

Definirana kot verjetnost, da bo programska oprema v predpisanem okolju v predpisanem času delovala brezhibno

Lahko se oceni z naključnim testiranjem

Testni podatki morajo v čim večji meri posnemati okolje za katerega je programska oprema namenjena

Vzorci uporabe sistema v prihodnosti so zajeti v opisu, ki ga imenujemo operativni profil

Cilji testiranja

Programska oprema deluje

Programska oprema ne deluje

Zmanjšati število odpovedi programske opreme

Zmanjšati ceno testiranja

Kaj je testni primer?

Testni primer je enostaven par

<vhod, pričakovan izhod>

Sistem brez stanj (Stateless): Prevajalnik je sistem brez stanj

Testni primeri so enostavni

Izhod je odvisen le od podanih vhodnih podatkov

Sistem s stanji: Bančni avtomat je sistem s stanji

Testni primeri niso več enostavni. Testni primer je lahko sestavljen iz zaporedja parov **<vhod, pričakovan izhod>**

Izhod je odvisen od trenutnega stanja sistema in podanih vhodnih podatkov

Primer bančnega avtomata:

< stanje na računu, 300.00€ >,

< dvig, “količina?” >,

< 200.00€, “200.00€” >,

< stanje na računu, 100.00€ >

Pričakovani rezultati

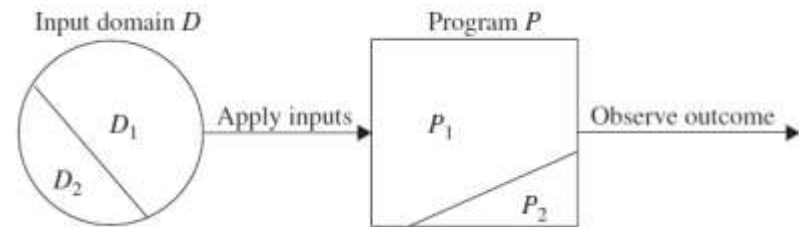
- ▶ Rezultat izvajanja programske opreme lahko zajema
 - ▶ Vrednost, ki jo izračuna program
 - ▶ Menjava stanja
 - ▶ Niz vrednosti, ki jih je potrebno interpretirati skupaj, da dobimo pravilen rezultat
- ▶ „Testni orakel“ je mehanizem, ki preveri pravilnost programskih izhodov
 - ▶ Generira pričakovane rezultate za podane testne podatke
 - ▶ Primerja pričakovane rezultate z dejanskimi rezultati testiranja implementacije

Koncept celovitega testiranja

- ▶ Celoviti in izčrpni testni postopki
 - “Na koncu te faze testiranja ni več nobenih nerazkritih napak”
- ▶ Celovito testiranje je praktično nemogoče za večino sistemov
 - ▶ Zaloga vrednosti vhodnih podatkov programske opreme je prevelika
 - ▶ Pravilni vhodni podatki
 - ▶ Nepravilni vhodni podatki
 - ▶ Programska oprema je lahko preveč kompleksna, da bi jo lahko v celoti stestirali
 - ▶ Po navadi je nemogoče poustvariti vsa možna okolja, v katerih bo sistem deloval

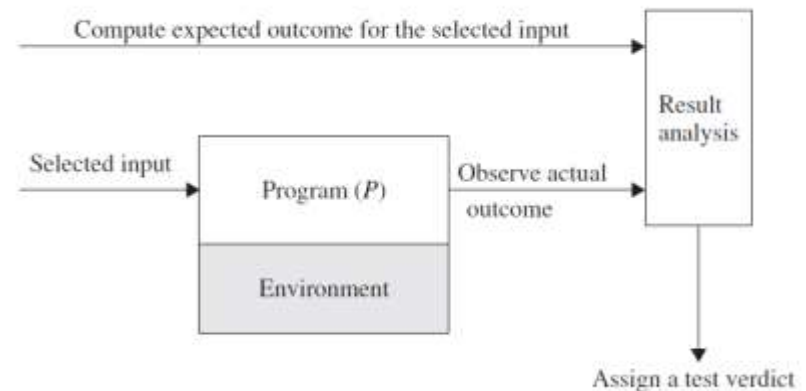
Osrednje vprašanje pri testiranju

- ▶ Množico vhodnih podatkov D razdelimo v dve podmnožici D_1 in D_2
- ▶ Za testiranje programa P izberemo podmnožico vhodnih podatkov D_1
- ▶ Mogoče je, da se z vrednostmi iz D_1 lahko testira le P_1 del programa P



Testne aktivnosti

- ▶ Določimo cilj našega testiranja
- ▶ Izberemo vhodne podatke
- ▶ Izračunamo (predvidimo) pričakovan rezultat
- ▶ Vzpostavimo testno okolje za izvajanje programa
- ▶ Izvedemo testne primere
- ▶ Analiziramo testne rezultate



Nivo testiranja

Testiranje modulov (Unit test)

- Testiramo posamezne programske enote, kot so procedure, metode ali funkcije posamezno

Testiranje integracije (Integration test)

- Module sestavimo da zgradimo večji podsistem za testiranje

Sistemsko testiranje (System test)

- Vsebuje širok spekter testov, npr. funkcionalni test, varnostni test, stabilnostni test, obremenitveni test itd.

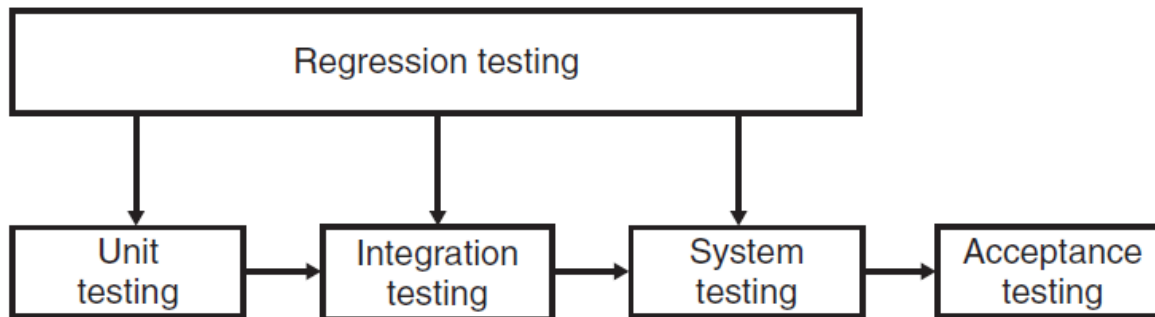
Prezemni test (Acceptance test)

- Preverimo ali sistem deluje v skladu z uporabniškimi zahtevami
- Izvajajo končni uporabniki v okolju podobnem produkcijskemu
- Dve vrsti prevzemnih testov
 - UAT - User Acceptance test
 - BAT - Business Acceptance test
- UAT: Izvajajo uporabniki, da ugotovijo ali sistem ustreza postavljenim kriterijem
- BAT: Izvede se na strani dobavitelja programske opreme

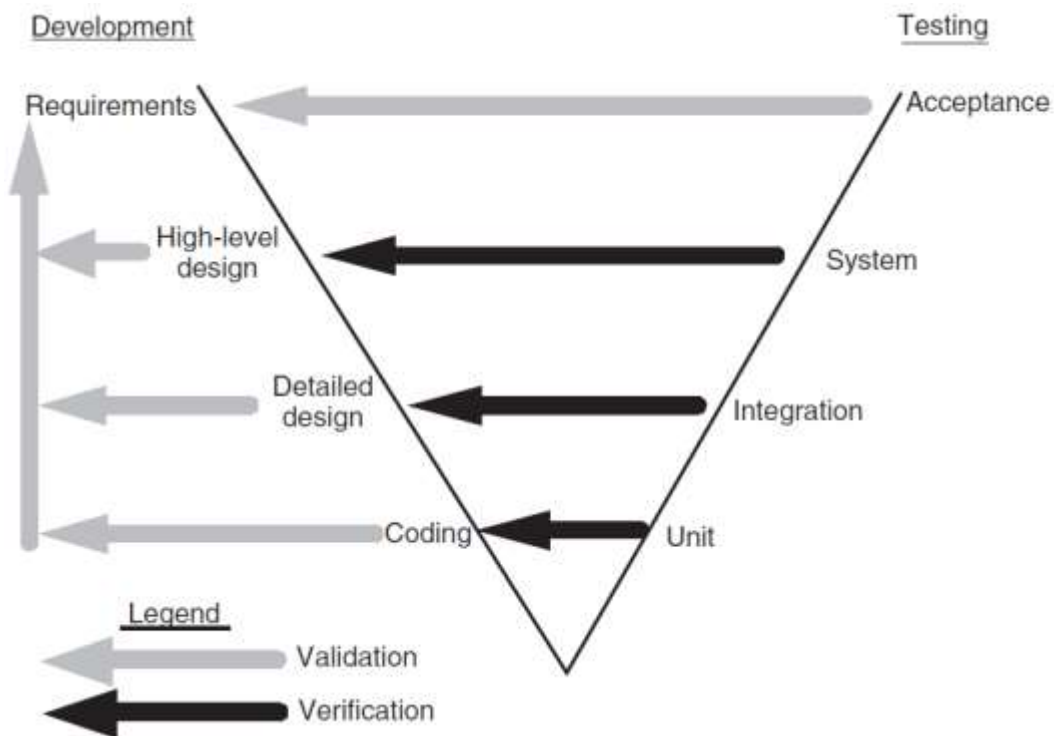
Regresijsko testiranje

- Nabor testov, ki se izvaja skozi celoten življenjski cikel, kadarkoli se spremeni kakšna komponenta sistema
- Testnih primerov ne spreminjamo, dodajamo
- Prepričamo se, da z novo verzijo nismo pokvarili programske kode

- Regresijsko testiranje v različnih fazah razvojnega cikla



- Razvoj in testiranje v obliki V modela



Viri podatkov za izbiro testnih metod

Specifikacija zahtev
in funkcionalnosti
(Requirement,
Functional
Specification)

Izvorna koda

Vhodna in izhodna
množica podatkov

Operacijski profil

Model napak

- Ugibanje napak
- Simbolično izvrševanje programov
- Mutacijska analiza

Testiranje po principu bele škatle

Testiranje bele škatle
t.i. strukturno
testiranje

Preveri izvorno kodo
s poudarkom na:

- Testiranje kontrole podatkov
- Testiranje toka podatkov

Testiranje kontrole
podatkov je
prehajanje kontrole
iz ene inštrukcije v
drugo

Tok podatkov je
propagacija vrednosti
iz ene spremenljivke
ali konstante v drugo
spremenljivko

Izvede se na
posameznih enota
programske opreme

Razvijalci izvedejo
strukturno testiranje
na enotami kode, ki
jo napišejo

Testiranje črne škatle t.i. funkcionalno testiranje

Preveri program, na način kakor je dosegljiv od zunaj

Določimo vhodne podatke in opazujemo zunanje vidne rezultate

Lahko se izvaja nad celotnim sistemom ali pa nad posameznimi programskimi enotami

Izvede se na zunanjih vmesnikih sistema

Izvede ga ločena skupina za zagotavljanje kvalitete programske opreme

Načrtovanje testiranja in testnih vzorcev

- ▶ Namen je pripraviti se na izvajanje testnih primerov
- ▶ Testni načrt sestavlja:
 - ▶ Ogrodje
 - ▶ Zbirka idej, dejstev in okoliščin znotraj katerih se bodo testi izvajali
 - ▶ Področje
 - ▶ Domena ali obseg testnih aktivnosti
 - ▶ Podrobnosti potrebnih virov
 - ▶ Zahtevan trud
 - ▶ Urnik aktivnosti
 - ▶ Vložek
- ▶ Testne cilje definirajo različni viri
- ▶ Vsak testnih primer je sestavljen iz kombinacije modularnih komponent imenovanih testni koraki
- ▶ Testni koraki sestavljajo zahtevnejše testne vzorce

Opazovanje in merjenje izvajanja testov

- ▶ Metrike za opazovanje izvajanja testov
- ▶ Metrike za opazovanje okvar
- ▶ Metrike za učinkovitost testnih primerov
 - ▶ Merilo za odkrivanje napak testnih orodij
 - ▶ Metriko uporabimo za izboljšanje načrtovanja testnih primerov
- ▶ Metrike za merjenje učinkovitosti testiranja
 - ▶ Število napak, ki jih odkrije stranka in jih niso odkrili razvijalci

Testna orodja in avtomatiziranje testiranja

- ▶ Povečana produktivnost testerjev
- ▶ Boljša pokritost z regresijskimi testi
- ▶ Krajše trajanje testnih faz
- ▶ Zmanjšana cena vzdrževanja programske opreme
- ▶ Povečana učinkovitost testnih primerov
- ▶ Testni primeri, ki jih avtomatiziramo so dobro definirani
- ▶ Velika množica testnih orodij in infrastrukture
- ▶ Strokovnjaki iz avtomatiziranja testnih vzorcev imajo izkušnje iz tega področja
- ▶ Proračun za nabavo testnih orodij je ponavadi že sprejet

Organizacija in vodenje testne skupine

- ▶ Najemanje in zadržanje dobrih testnih inženirjev je zahtevna naloga
- ▶ Najpomembnejši mehanizem pri najemanju strokovnjakov je intervju
- ▶ Potrebne so različne skupine testerjev za testiranje na različnih nivojih
- ▶ Management se mora zavedati, da je testiranje ravno tako pomemben del v razvojnem ciklu produkta in zagotavljanja kvalitete, kot sam razvoj
- ▶ Testne inženirje je potrebno obravnavati kot profesionalce in pomemben del skupine, ki razvija oz. proizvaja kvaliteten produkt

Trendi v 2022

- ▶ TestOps: testiranje v povezavi z DevOps principi in kontinuirano integracijo ter predajo PO
- ▶ Posvečanje več pozornosti QA veščinam in kompetencam
- ▶ Reorganizacija QA v timsko aktivnost, kar ne pomeni, da v timu ni več QA specialistov
- ▶ Uvedba UI v testiranje
- ▶ Še večji poudarek uporabniški izkušnji

Trendi v 2023

- ▶ Testiranje v oblaku
- ▶ Avtomatizirano testiranje
- ▶ Neprekinjeno testiranje in testiranje delovanja
- ▶ Umetna inteligenca (AI)
- ▶ Testiranje mobilne aplikacije
- ▶ DevOps in varnostno testiranje

Trendi v 2024

Brez-kodna avtomatizacija testiranja, ki domenskim ekspertom in uporabnikom, brez znanja računalništva omogoča avtomatizacijo in sodelovanje v testiranju

Zagotavljanje kvalitete s pomočjo UI, kar predstavlja še korak naprej od tradicionalne avtomatizacije testov

Razumljiva UI v testiranju

„Self healing“ - avtotsko odkrivanje in popravljanje napak

„Shift - left“ - sprotno avtomatsko testiranje

Napredno upravljanje s podatki za testiranje

Testiranje kvantnega računanja

QAOos - integracija DevOps in QA (Quality assurance)

Kombinacija manualnega in avtomatskega testiranja